

CROP ROTATION ADVISOR BASED ON SOIL NUTRIENT DEPLETION TRACKING

BY

TEAM 6

R HARIKA (192572140)

SHAHANA (192524479)

CASSANDRA MAYA NEELALI (192521073)

Source Code:

```
#include "RuleEngine.h"

#include <algorithm> // std::sort

RuleEngine::RuleEngine() {}

double RuleEngine::scoreCrop(const Plot& plot, const Crop& crop,
std::vector<std::string>& reasons) const {

    double score = 0.0;

    // --- Reward crops that replenish whichever nutrients are currently low -
    --
    if (plot.getN() < DEFICIT_THRESHOLD && crop.getNDelta() > 0) {
        score += crop.getNDelta() * 2.0;
        reasons.push_back("replenishes Nitrogen, which is currently low on
this plot");
    }

    if (plot.getP() < DEFICIT_THRESHOLD && crop.getPDelta() > 0) {
        score += crop.getPDelta() * 2.0;
        reasons.push_back("replenishes Phosphorus, which is currently low on
this plot");
    }
}
```

```

    if (plot.getK() < DEFICIT_THRESHOLD && crop.getKDelta() > 0) {
        score += crop.getKDelta() * 2.0;
        reasons.push_back("replenishes Potassium, which is currently low on
this plot");
    }

    // --- Mild penalty for crops that deplete nutrients that are already fine
    ---

    score += crop.getNDelta() * 0.5;
    score += crop.getPDelta() * 0.5;
    score += crop.getKDelta() * 0.5;

    // --- Penalize repeating the same family as recent history (rotation
risk) ---

    const std::vector<Crop>& history = plot.getHistory();

    int lookback = 2; // how many recent seasons we check

    int start = (int)history.size() > lookback ? (int)history.size() -
lookback : 0;

    for (int i = start; i < (int)history.size(); i++) {
        if (history[i].getFamily() == crop.getFamily()) {
            score -= 15.0;
            reasons.push_back(
                "same family (" + crop.getFamily() + ") as a recently planted
crop \u2014 rotation risk"
            );
            break; // one penalty is enough, don't stack it per matching
season
        }
    }
}

```

```

        return score;
    }

std::vector<Recommendation> RuleEngine::recommend(const Plot& plot, const
std::vector<Crop>& catalog) const {

    std::vector<Recommendation> results;

    for (const Crop& crop : catalog) {

        std::vector<std::string> reasons;

        double score = scoreCrop(plot, crop, reasons);

        results.push_back(Recommendation(crop.getName(), score, reasons));

    }

    // sort best-first
    std::sort(results.begin(), results.end(),

        [](const Recommendation& a, const Recommendation& b) {

            return a.score > b.score;

        });

    return results;
}

#include <iostream>

#include <sstream>

#include <fstream>

#include <string>

#include <vector>

#include <ctime>

#include "../lib/json.hpp"

```

```

#include "Crop.h"

#include "Plot.h"

#include "RuleEngine.h"

#include "MonocultureDetector.h"

using json = nlohmann::json;

// --- small helper: current timestamp as a string, for the log file ---
std::string nowTimestamp() {
    std::time_t t = std::time(nullptr);
    char buf[32];
    std::strftime(buf, sizeof(buf), "%Y-%m-%d %H:%M:%S", std::localtime(&t));
    return std::string(buf);
}

// --- build a Crop object from one JSON entry in "crop_catalog" ---
Crop cropFromJson(const json& j) {
    return Crop(
        j.at("name").get<std::string>(),
        j.at("family").get<std::string>(),
        j.at("n_delta").get<int>(),
        j.at("p_delta").get<int>(),
        j.at("k_delta").get<int>()
    );
}

int main() {
    // 1. Read ALL of stdin into one string
    std::stringstream buffer;

```

```

buffer << std::cin.rdbuf();

json request;

try {
    request = json::parse(buffer.str());
} catch (const std::exception& e) {
    json err = { {"status", "error"}, {"message", std::string("invalid
JSON input: ") + e.what()} };

    std::cout << err.dump();

    return 1; // genuine crash-level error, not a handled one
}

try {
    // 2. Build the Plot from request.soil + request.plot_id

    int plotId = request.at("plot_id").get<int>();

    json soil = request.at("soil");

    Plot plot(plotId, "plot_" + std::to_string(plotId),
               soil.at("n").get<int>(), soil.at("p").get<int>(),
soil.at("k").get<int>());

    // 3. Build the crop catalog (vector<Crop>) from request.crop_catalog

    std::vector<Crop> catalog;

    for (const auto& c : request.at("crop_catalog")) {
        catalog.push_back(cropFromJson(c));
    }

    // 4. Build Plot's history from request.history (matching names
against the catalog)

    for (const auto& h : request.at("history")) {
        std::string cropName = h.at("crop").get<std::string>();

```

```

        for (const Crop& c : catalog) {
            if (c.getName() == cropName) {
                plot.addHistory(c);
                break;
            }
        }
    }

    // 5. Find the Crop object for "planted_crop" (the one just logged)
    std::string plantedCropName =
request.at("planted_crop").get<std::string>();

    Crop plantedCrop;
    bool found = false;
    for (const Crop& c : catalog) {
        if (c.getName() == plantedCropName) {
            plantedCrop = c;
            found = true;
            break;
        }
    }

    if (!found) {
        json err = { {"status", "error"}, {"message", "unknown crop '" +
plantedCropName + "' not in catalog"} };

        std::cout << err.dump();

        return 0; // handled error, not a crash
    }

    // 6. Run the monoculture check against history (BEFORE adding
plantedCrop into it)

    MonocultureDetector detector;

```

```

    bool flagged = detector.isRisk(plot, plantedCrop);

    std::string reason = detector.explain(plot, plantedCrop);

    int consecutive = detector.consecutiveCount(plot, plantedCrop);

    // 7. Apply the planted crop's effect on soil, and log it into plot
history
    plot.applyDeltas(plantedCrop.getNDelta(), plantedCrop.getPDelta(),
plantedCrop.getKDelta());

    plot.addHistory(plantedCrop);

    // 8. Get ranked recommendations for NEXT season, based on updated
soil + updated history
    RuleEngine engine;

    std::vector<Recommendation> recs = engine.recommend(plot, catalog);

    // 9. Build the response JSON
    json response;

    response["status"] = "ok";

    response["plot_id"] = plotId;

    response["season_logged"] = request.at("current_season").get<int>();
    response["crop_logged"] = plantedCropName;

    response["monoculture_alert"] = {
        {"flagged", flagged},
        {"reason", reason},
        {"consecutive_count", consecutive}
    };

    response["projected_soil"] = {
        {"n", plot.getN()},

```

```

        {"p", plot.getP()},
        {"k", plot.getK()}
    };

    json recArray = json::array();
    for (const Recommendation& r : recs) {
        recArray.push_back({
            {"crop", r.cropName},
            {"score", r.score},
            {"reasons", r.reasons}
        });
    }

    response["recommendations"] = recArray;

    // 10. Append to local log file (this is your file-handling
requirement, alive and real)

    std::ofstream logFile("agri_engine.log", std::ios::app);
    if (logFile.is_open()) {
        logFile << "[" << nowTimestamp() << "]" plot " << plotId
            << " planted " << plantedCropName
            << " | monoculture_flagged=" << (flagged ? "true" :
"false") << "\n";
        logFile.close();
    }

    // 11. Print the response JSON to stdout – this is ALL that should go
to stdout

    std::cout << response.dump();

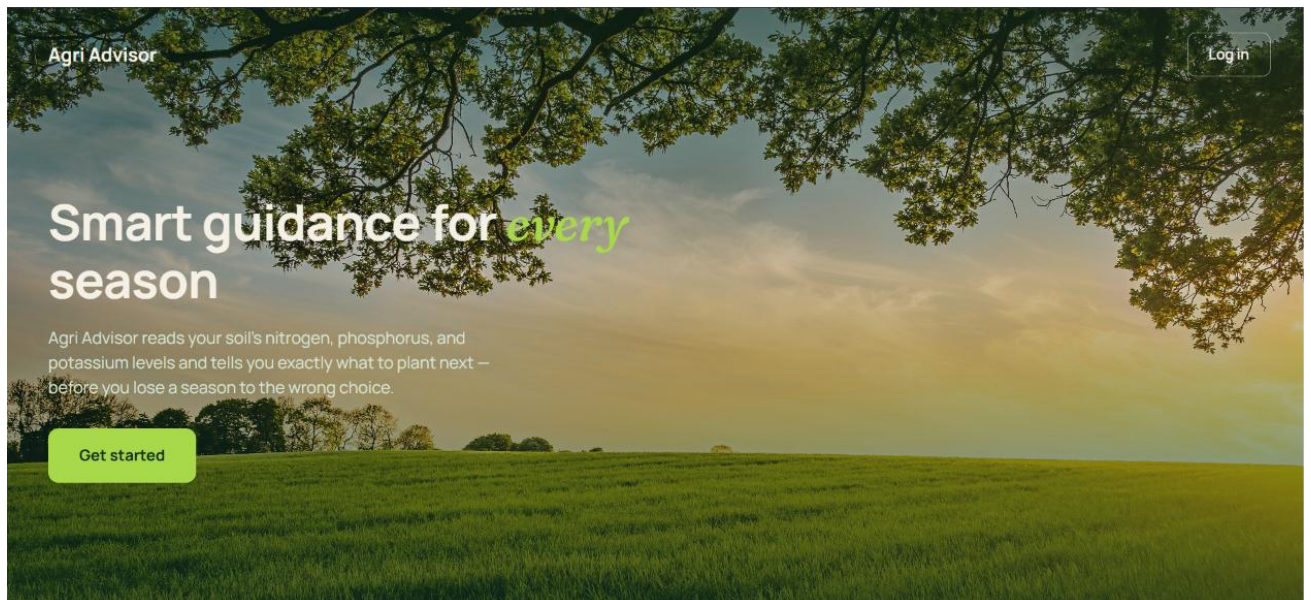
    return 0;

```



```
    } catch (const std::exception& e) {  
        json err = { {"status", "error"}, {"message", std::string("processing  
error: ") + e.what()} };  
  
        std::cout << err.dump();  
  
        return 0;  
    }  
}
```

Implementation Screenshots:



Your Plots

Click a plot to see its soil trends, planting history, and get a new recommendation.

plot_1

NEEDS ATTENTION

Add a phosphorus-based fertilizer and a potassium source to balance the nutrients and support healthy crop growth.



plot_2

HEALTHY



plot_3

HEALTHY



Current Soil Levels

NITROGEN (N)

151

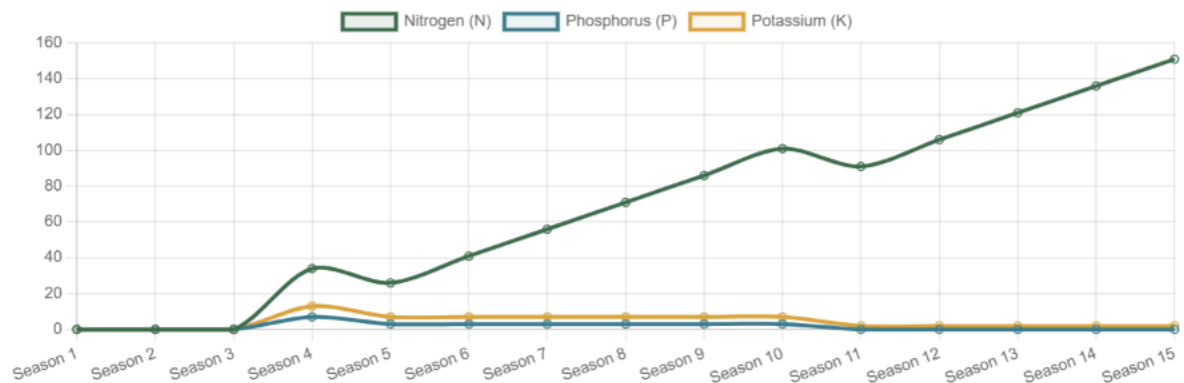
PHOSPHORUS (P)

0

POTASSIUM (K)

2

Soil Trend Over Time



Agri Advisor — Plot Report

plot_1 · Planted Lentils · Season 17 · Generated 2026-09-02 19:04

Monoculture warning: Legume-family crops planted 6 consecutive seasons (including this one) — soil depletion and pest/disease risk rise sharply.

Current Soil Levels

Nitrogen (N)	Phosphorus (P)	Potassium (K)
166	0	2

Projected Soil Levels After Planting

Nitrogen (N)	Phosphorus (P)	Potassium (K)
181	0	2

Crop Recommendations

Lentils — Score: -7.5 same family (Legume) as a recently planted crop — rotation risk
Maize — Score: -9
Rice — Score: -10

Season History

Season 1: Planted Maize (N: 0, P: 0, K: 0) — 2026-08-26 23:28:45
Season 2: Planted Lentils (N: 0, P: 0, K: 0) — 2026-08-26 23:41:13
Season 3: Planted Maize (N: 0, P: 0, K: 0) — 2026-08-26 23:47:49
Season 4: Planted Lentils (N: 34, P: 7, K: 13) — 2026-08-27 00:07:00
Season 5: Planted Maize (N: 26, P: 3, K: 7) — 2026-08-27 00:15:36
Season 6: Planted Lentils (N: 41, P: 3, K: 7) — 2026-08-27 09:23:32
Season 7: Planted Lentils (N: 56, P: 3, K: 7) — 2026-08-27 10:13:07
Season 8: Planted Lentils (N: 71, P: 3, K: 7) — 2026-08-27 10:41:25